

Laboratório de Banco de Dados

Profa. Msc. Esteic Janaina
Santos Batista

Módulo 2 - Utilização prática de um SGBD usando Funções

Unidade 1 - Funções e Procedimentos armazenados



UFMS

AGEAD

Agência de Educação
Digital e a Distância

Funções e procedimentos no PostgreSQL

Neste módulo, vamos:

- Explorar como funções e procedimentos armazenados podem encapsular regras de negócio diretamente no banco de dados, promovendo a reutilização e a eficiência.
- Entender o que são essas estruturas, suas utilidades práticas e por que são fundamentais para o gerenciamento de sistemas robustos de banco de dados.

Definição

- **Função:** um bloco de código SQL que executa operações no banco de dados, retorna um valor e pode ser chamado por queries.
- **Procedimento armazenado:** um bloco de código SQL que executa uma série de operações, mas não necessariamente retorna um valor.

Definição

- **Função:** um bloco de código SQL que executa operações no banco de dados, retorna um valor e pode ser chamado por *queries*.

equivalência no PostgreSQL

- **Procedimento armazenado:** um bloco de código SQL que executa uma série de operações, mas não necessariamente retorna um valor.
- **Exemplo:** calcular total de créditos de um aluno.

Importância das funções e procedimentos

**Reutilização
de Código**

Escrever uma vez, usar em várias partes do sistema.

Eficiência

Funções executam diretamente no servidor, eliminando a necessidade de transferir dados.

**Manutenção
Centralizada**

Qualquer alteração é feita diretamente no banco de dados, afetando todas as partes do sistema que usam aquela função.

Exemplo de aplicação

- Alunos se inscrevem em cursos a partir de um sistema da universidade.
- **Requisitos:** número máximo de créditos permitidos, número mínimo de disciplinas por semestre, entre outros.
- **Sem encapsular:** reescrever todas as regras e validações todas as vezes que precisássemos realizar uma inscrição.
- **Solução:** criar uma função no banco de dados chamada `inscreveAluno`, que verifica se o aluno cumpre todos os requisitos para se inscrever e, em seguida, realiza a inscrição no curso.

Sintaxe de uma função

```
CREATE FUNCTION function_name(param1 type1, param2 type2)
```

```
RETURNS return_type AS $$
```

```
BEGIN
```

```
-- Corpo da função
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```


Sintaxe de uma função

```
CREATE FUNCTION somar_numeros(a INT, b INT)
```

```
RETURNS INT AS $$
```

```
BEGIN
```

```
    RETURN a + b;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Sintaxe de uma função

- `CREATE FUNCTION` `somar_numeros(a INT, b INT)`: Estamos criando uma função chamada `somar_numeros` que recebe dois números inteiros (`a` e `b`).
- `RETURNS INT`: A função vai retornar (devolver) um número inteiro.
- `AS BEGIN...END`: Este é o corpo da função. Entre o `BEGIN` e o `END`, colocamos a lógica que vai ser executada.
- `RETURN a + b`: A função simplesmente pega os valores de `a` e `b`, soma esses números e devolve o resultado.
- `LANGUAGE plpgsql`: A linguagem usada para escrever a função.

Tipos de retorno em funções no PostgreSQL

- No PostgreSQL, as funções podem retornar diferentes tipos de dados, dependendo da necessidade da aplicação e da lógica que você deseja implementar.
- Cada tipo de retorno tem sua aplicação específica.

Tipos de retorno em funções no PostgreSQL

Escalares

- **Definição:** são valores simples, como inteiros, números de ponto flutuante, texto, etc. Esse é o tipo de retorno mais comum e simples.
- **Quando utilizar:** usamos funções escalares quando queremos que a função execute algum cálculo ou transformação e devolva um único valor como resultado.

Tipos de retorno em funções no PostgreSQL

Escalares

```
CREATE FUNCTION calcular_desconto(preco NUMERIC, desconto NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
    RETURN preco * (1 - desconto / 100);
END;
$$ LANGUAGE plpgsql;

-- Chamada da função:
SELECT calcular_desconto(100, 10); -- Retorna 90 (valor com desconto)
```

Tipos de retorno em funções no PostgreSQL

Tabelas (SETOF)

- **Definição:** funções que retornam conjuntos de registros, ou seja, várias linhas de uma tabela, ao invés de um único valor.
- **Quando utilizar:** essas funções são úteis quando queremos que a função se comporte como uma tabela e retorne múltiplas linhas, como em consultas que podem retornar resultados dinâmicos.

Tipos de retorno em funções no PostgreSQL

Tabelas (SETOF)

```
CREATE FUNCTION buscar_alunos_por_curso(curso_nome TEXT)
RETURNS SETOF cursos AS $$
BEGIN
    RETURN QUERY SELECT * FROM cursos WHERE nome_curso = curso_nome;
END;
$$ LANGUAGE plpgsql;

-- Chamada da função:
SELECT * FROM buscar_alunos_por_curso('Engenharia');
```

Tipos de retorno em funções no PostgreSQL

Arrays

- **Definição:** funções que retornam *arrays* (vetores) de valores. *Arrays* podem conter qualquer tipo de dados, como inteiros, texto ou até mesmo registros.
- **Quando utilizar:** quando queremos retornar uma coleção de valores de uma única função, mas organizados em uma estrutura de *array*.

Tipos de retorno em funções no PostgreSQL

Arrays

```
CREATE TABLE alunos (  
    id SERIAL PRIMARY KEY,  
    nome TEXT NOT NULL,  
    curso TEXT NOT NULL,  
    nota INTEGER  
);  
  
-- Inserindo dados na tabela  
INSERT INTO alunos (nome, curso, nota) VALUES  
('João Silva', 'Engenharia', 85),  
('Maria Souza', 'Medicina', 90),  
('Carlos Andrade', 'Engenharia', 75),  
('Ana Lima', 'Direito', 88),  
('Paulo Mendes', 'Medicina', 92);
```

Tipos de retorno em funções no PostgreSQL

Arrays

```
CREATE FUNCTION listar_notas_por_curso(curso_nome TEXT)
RETURNS INTEGER[] AS $$
DECLARE
    notas INTEGER[];
BEGIN
    -- Seleciona todas as notas dos alunos do curso e armazena no array
    SELECT array_agg(nota) INTO notas
    FROM alunos WHERE curso = curso_nome;

    -- Retorna o array de notas
    RETURN notas;
END;
$$ LANGUAGE plpgsql;
```

Tipos de retorno em funções no PostgreSQL

Arrays

- Agora, você pode chamar a função e verificar as notas dos alunos de um curso específico. Por exemplo:

```
SELECT listar_notas_por_curso('Medicina');
```

- A consulta anterior deve retornar algo assim:

```
listar_notas_por_curso
```

```
{90,92}
```

(1 row)

Tipos de retorno em funções no PostgreSQL

Record (Registro)

- **Definição:** funções que retornam um único registro, ou seja, uma linha de dados com múltiplas colunas. O tipo **RECORD** é usado quando não sabemos previamente a estrutura exata do retorno.
- **Quando utilizar:** quando queremos retornar uma linha específica contendo múltiplos valores, mas não queremos definir uma tabela específica para o retorno.

Tipos de retorno em funções no PostgreSQL

Record (Registro)

```
CREATE FUNCTION buscar_informacoes_aluno(aluno_id INT)
RETURNS RECORD AS $$
DECLARE
    resultado RECORD;
BEGIN
    -- Seleciona as informações do aluno com o ID fornecido
    SELECT nome, curso, nota INTO resultado
    FROM alunos WHERE id = aluno_id;

    -- Retorna o resultado
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;
```

Tipos de retorno em funções no PostgreSQL

Record (Registro)

- Para chamar uma função que retorna um RECORD, precisamos especificar as colunas do retorno na consulta, porque o PostgreSQL precisa saber quais colunas estarão presentes no RECORD.

```
SELECT * FROM buscar_informacoes_aluno(2) AS (nome TEXT,  
curso TEXT, nota INT);
```

Chamando funções no PostgreSQL

- **Sintaxe:** Para chamar uma função, usamos o comando SELECT seguido pelo nome da função e seus parâmetros entre parênteses. **Exemplo:**

```
SELECT nome_da_funcao(param1, param2);
```

- Isso faz com que o PostgreSQL execute a função e retorne o valor calculado ou os dados que a função está programada para retornar.

Procedimentos (*Procedures*)

- Procedimentos não retornam um valor diretamente, como as funções.
- Eles são usados para executar tarefas que não exigem um retorno de valor, como inserções, atualizações ou execuções de operações complexas.

Procedimentos (*Procedures*)

```
CREATE PROCEDURE adicionar_aluno(nome TEXT, curso TEXT)
```

```
LANGUAGE plpgsql AS $$
```

```
BEGIN
```

```
    INSERT INTO alunos (nome, curso) VALUES (nome, curso);
```

```
END;
```

```
$$;
```

Chamando procedimentos (*Procedures*)

```
CALL adicionar_aluno('João', 'Física');
```

Alterar função

- Você pode precisar alterar uma função para modificar sua lógica, mudar os tipos de parâmetros ou ajustar seu tipo de retorno.
- No PostgreSQL, para modificar o código de uma função (corpo), usa-se **CREATE OR REPLACE FUNCTION**.
- Para ajustar propriedades, como o proprietário, usa-se **ALTER FUNCTION**.

Alterar função

```
ALTER FUNCTION minha_funcao(INTEGER)
```

```
OWNER TO novo_proprietario;
```

```
CREATE OR REPLACE FUNCTION minha_funcao(param1 INT)
```

```
RETURNS INT AS $$
```

```
BEGIN
```

```
    RETURN param1 * 2;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Exclusão de funções

- Às vezes, uma função pode não ser mais necessária, ou pode ser necessário remover uma função para evitar que ela seja usada por outros sistemas ou por motivos de segurança.
- Para excluir uma função no PostgreSQL, você usa o comando **DROP FUNCTION**.
- É importante especificar os tipos de parâmetros que a função aceita ao excluí-la, pois podem existir várias funções com o mesmo nome, mas com parâmetros diferentes.

Exclusão de funções

DROP FUNCTION.

- Excluir uma função com um parâmetro inteiro:

DROP FUNCTION minha_funcao(INTEGER);

- Excluir uma função que não aceita parâmetros:

DROP FUNCTION minha_funcao();

Exclusão de procedimentos

- O comando para excluir um procedimento é DROP
PROCEDURE.

DROP PROCEDURE meu_procedimento(INTEGER, TEXT);

Cuidado: Ao excluir uma função ou procedimento, qualquer código que dependa dele deixará de funcionar. Se houver dependências, é importante verificar antes de executar o DROP.

POSTGRESQL. **Tradução da Documentação para o Português do Brasil.** Disponível em:
<https://link.ufms.br/8Aaxp>. Acesso em: 05 maio 2024. **Trecho a ser lido: Capítulo 31. Seção 31.4**

SILBERSCHATZ, Abraham; Korth, Henry F.; Sudarshan, S. **Sistema de Banco de Dados.** Rio de Janeiro: GEN LTC, 2020. ISBN 9788595157552. [Disponível na Biblioteca Digital da UFMS](#) **Trecho a ser lido: Capítulo 5. Seção 5.2.**

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](#).

